CNN Architectures

ECON 8310: Business Forecasting — Lecture 8

Department of Economics

University of Nebraska at Omaha

October 13, 2026

Lecture Outline

1 Convolution: The Core Idea

2 Pooling and Dropout

3 CNN Architecture Evolution

4 1D CNNs for Time Series

5 Key Takeaways

Convolution: The Core Idea

Look at a small window, and look for the same thing everywhere. Two ideas, and they are the whole architecture.

Why Convolution?

Start with why a fully connected layer fails on structured input. A 224×224 image has 50,176 pixels, so a single hidden layer of 1,000 neurons needs **50 million weights** — for one layer. Lecture 7's network had 22,657 parameters in total.

Worse than the cost is the waste. That layer ignores **spatial structure**: it has no notion that neighboring pixels are related and distant ones usually are not. And it does not **generalize position** — a pattern learned in the top-left corner is stored separately from the same pattern in the bottom-right.

> A convolutional layer fixes both with two ideas. **Local connectivity:** each neuron sees only a small **receptive field**. **Weight sharing:** the same filter is applied at every position. Far fewer parameters, and a pattern learned anywhere is recognized everywhere.

The analogy that makes it stick: a reader does not memorize the letter "A" separately for every position on the page. They learn it once and apply it everywhere. A filter does exactly that.

How a Convolutional Layer Works

For a sequence, a 1D convolution slides a filter of length K along the series and takes a dot product at each position:

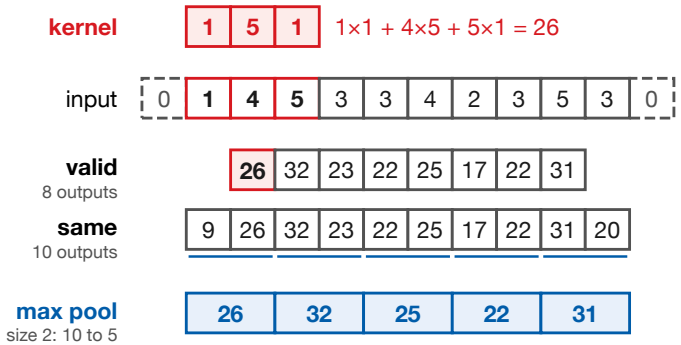
$$(f * g)[t] = \sum_{k=0}^{K-1} f[k] \cdot g[t-k]$$

Kernel size	Filter length K. Larger means a wider receptive field — more context per output, and more parameters.
Stride	How far the filter moves between positions. Stride 2 halves the output length.
Padding	Zeros at the edges. `padding='same'` keeps output length equal to input length.
Filters	How many different patterns to learn. This becomes the number of output channels.

Each filter learns one **local pattern** — an edge, a spike, a seasonal dip. Its output is a **feature map** whose high values mark where that pattern was found.

Convolution, Step by Step

A length-10 series and a width-3 filter $[1, 5, 1]$. Slide the filter along and take a dot product at each position.



Valid keeps only full overlaps: $10 - 3 + 1 = 8$ outputs. **Same** pads a zero at each end and keeps all 10. Max pooling then keeps the larger of each pair.

What a Filter Sees

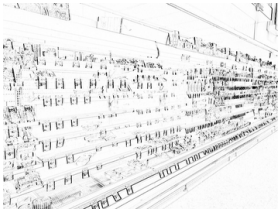
The same operation in two dimensions: a 3×3 filter slides over every pixel of a photograph. Two hand-set filters find two different kinds of structure.

the photo



Photo: Kurt Kaiser, 18 March 2020 (CC0)

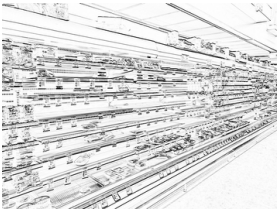
vertical edges



left vs right

-1	0	+1
-2	0	+2
-1	0	+1

horizontal edges



above vs below

-1	-2	-1
0	0	0
+1	+2	+1

18 March 2020: shelves emptied by a shock no forecast saw coming — more in Week 16.

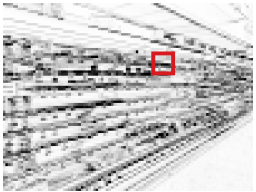
Pooling and Dropout

Pooling shrinks the feature map and buys robustness — at a price that matters more for forecasting than for images.

Pooling: Downsampling Feature Maps

Pooling summarizes each small region of a feature map with one number: max pooling keeps the largest, average pooling the mean. A 2×2 max pool quarters the values.

horizontal-edge map



the 8 × 8 patch

1	1	1	1	1	1	1	1
2	1	**2**	2	**2**	2	**2**	2
1	1	2	2	**2**	2	**2**	2
4	3	**3**	2	1	1	1	1
6	6	5	**7**	**9**	7	**6**	3
5	4	3	4	4	5	5	5
0	0	**3**	0	1	2	2	2
2	2	3	3	**3**	2	1	**3**

2 × 2 max pool

2	**2**	**2**	**2**
4	**3**	**2**	**2**
6	**7**	**9**	**6**
2	**3**	**3**	**3**

It buys **fewer parameters downstream** and **translation invariance**: shift the input slightly and the pooled output barely changes. The shelf edge above survives the pool.

A feature for images, a problem for forecasting. A demand spike that moved three weeks later is what you are predicting, not noise. Pooling discards *where* a pattern occurred — we measure the cost later in this lecture.

CNN Architecture Evolution

Thirty years of architectures, and three ideas that survived.

LeNet to VGG: Depth and Small Filters

LeNet-5 (LeCun et al. 1998) established the template in 1998: alternating convolution and pooling layers, then fully connected layers at the end. It had roughly 60,000 parameters and read handwritten digits on cheques.

VGG (Simonyan and Zisserman 2015) scaled that idea up in 2014 and made one sharp simplification: use *only* 3×3 filters, but stack many of them. Two stacked 3×3 layers see the same receptive field as one 5×5, with fewer parameters and an extra nonlinearity in between.

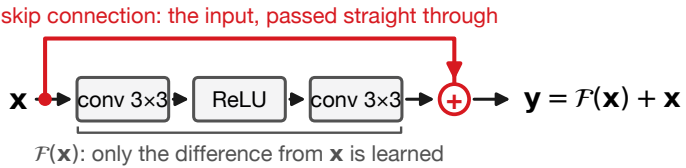
> **The lesson that carried:** depth built from small filters beats width built from large ones. It is cheaper in parameters and more expressive, and it is why almost every modern architecture is deep and narrow rather than shallow and wide.

VGG's cost was its size — around 138 million parameters, most of them in the final fully connected layers. That is the problem the next two architectures attacked.

Inception and ResNet: Depth That Trains

Inception (Szegedy et al. 2015) asked why a layer should commit to one filter size: it runs 1×1, 3×3 and 5×5 convolutions *in parallel* and lets the network choose the scale, with 1×1 convolutions cutting channels first to keep cost down.

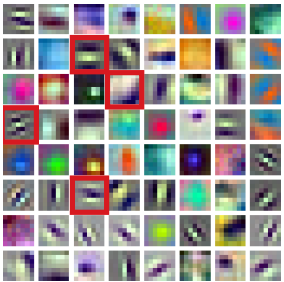
ResNet (He et al. 2016) solved a different problem. Past a certain depth, deeper networks got *worse* — not from overfitting, but because gradients degraded on their way back through many layers. The fix is a **residual** or skip connection:

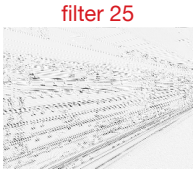


The $+\mathbf{x}$ gives the gradient a direct path backward. That one change made networks of 100+ layers trainable, and skip connections now appear in nearly every deep architecture — including the Transformers in Lecture 9.

What a Trained CNN Learns

ResNet-18, trained on 1.2 million labelled images. Nobody designed its first layer — these are the 64 filters it arrived at, and what four of them find in the same photograph.











Many look like the edge filters we set by hand; the rest detect colour. **Learned, not engineered** — the same promise a 1D CNN makes for a time series.

What Carries Over to Time Series

These architectures were built for images, on problems that look nothing like weekly demand forecasting. Three of their ideas transfer anyway.

Small stacked filters	A width-3 filter twice sees six weeks of context with far fewer weights than one width-6 filter.
Skip connections	Same fix for the same problem — gradients that fade through depth.
Learned features	You stop hand-engineering lag windows and let filters find the local shapes that matter.

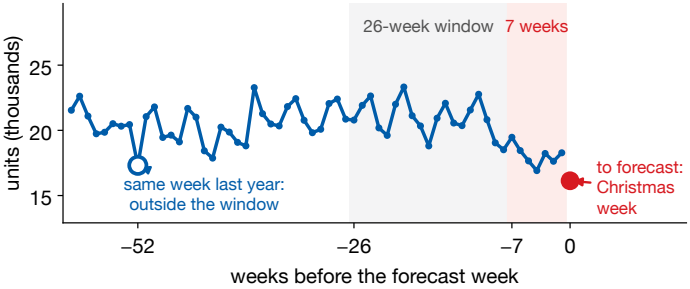
One idea does not transfer, and it is the one we flagged: pooling for translation invariance. An image classifier should not care where the cat is. A forecaster usually should care where the spike was.

1D CNNs for Time Series

The same machinery, one dimension, and an honest measurement.

Why Use a 1D CNN for Forecasting?

A 1D CNN takes a **window** of the recent past — 26 weeks of units, SNAP days, events and price — and predicts the next week. You stop choosing lags: the filters learn which local shapes matter. It is far cheaper than a dense network (weight sharing again), and the same filters serve *every* store-category at once.



The catch is the receptive field. Stacked width-5 and width-3 filters see seven weeks; annual seasonality is 52 away. Add dilation, or feed the annual lag in as its own channel.

1D CNN Forecasting Pipeline in PyTorch

Window in, one forecast out

```
import torch.nn as nn

model = nn.Sequential(

nn.Conv1d(n_channels, 32, kernel_size=5), nn.ReLU(),

nn.Conv1d(32, 64, kernel_size=3),       nn.ReLU(),

nn.AdaptiveAvgPool1d(1),   # collapses the time axis

nn.Flatten(),

nn.Linear(64, 1))
```

Input shape is `(batch, channels, time)` — note that channels come *before* time, which is the transpose of how the data naturally sits in a dataframe.

Everything from Lecture 7 still applies: Adam with weight decay, `zero_grad()` every batch, `shuffle=False`, and a split by time rather than at random.

`AdaptiveAvgPool1d(1)` averages across the whole time axis. Watch what that does on the next slide.

Does It Actually Work?

Fitted on our weekly panel — 26-week windows, four channels, 30 store-category series, the same test block as Homework 4.

Model	Test RMSE	Parameters
LASSO (Homework 4)	**744**	46
XGBoost (Homework 4)	781	—
Random forest (Homework 3)	899	—
1D CNN, flatten head	1,032	8,161
1D CNN, global max pool	1,418	6,945
1D CNN, global average pool	1,534	6,945
Seasonal naive	2,152	0

Every CNN variant loses to a 46-coefficient linear model — a seven-week receptive field cannot see the annual lag LASSO was handed directly.

And swapping average pooling for `Flatten`, which keeps *where* each pattern occurred, is worth about **500 RMSE** — three seeds each, against a spread of about 80.

Key Takeaways

What convolution is for, and what it is not for.

CNN Architecture Summary

Architecture	Year	The idea that survived
LeNet-5	1998	Conv–pool–conv–pool–dense: the template
VGG	2014	Only 3×3 filters, stacked deep
Inception	2014	Parallel filter sizes; 1×1 to cut channels
ResNet	2015	Skip connections make real depth trainable
1D CNN	—	The same machinery along a time axis

Reach for a 1D CNN when local shape matters more than exact position, when you have many parallel series, or as a cheap feature extractor feeding something else.

Do not reach for one when the dominant signal is at a long lag, when you have well engineered tabular features already, or when a stakeholder needs to read the model. On the evidence above, that covers most of what we forecast in this course.

Lecture 8: Key Takeaways

1. **Convolution** is local connectivity plus weight sharing. A pattern is learned once and detected everywhere, at a fraction of the parameters.
2. **Pooling** buys translation invariance — which an image classifier wants and a forecaster usually does not. Measured cost here: about 500 RMSE.
3. The architecture history reduces to three transferable ideas: **small filters stacked deep**, **skip connections** for trainable depth, and **learned** rather than hand-engineered features.
4. A **1D CNN** takes a window across channels and predicts the next step. Its weakness is the **receptive field**: seven weeks of context cannot see an annual lag.
5. On our panel every CNN variant **lost to a 46-coefficient LASSO**. Architecture has to fit the problem; it is not a ladder you climb.

Next: RNNs and LSTMs — architectures built to carry state through time, which is what the receptive field could not do.

References I

- He, Kaiming et al. (2016). "Deep Residual Learning for Image Recognition". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- LeCun, Yann et al. (1998). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.
- Simonyan, Karen and Andrew Zisserman (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition". In: *International Conference on Learning Representations (ICLR)*.
- Szegedy, Christian et al. (2015). "Going Deeper with Convolutions". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1–9.